

RAG System Engineering

From Intuition to Mathematical Foundations

yuyaow@bu.edu

- 1 What Is RAG?
- 2 From Text to Vectors: Embeddings
- 3 Retrieval as a Mathematical Problem
- 4 Training Embedding Models
- 5 The Generation Step
- 6 Open Research Problems & Contributions
- 7 Conclusion

The Core Problem

Ask a language model about your company's internal policy. . .

"What is our company's leave policy?"

[!] The Problem

The model's knowledge is **frozen at training time**.

It has never seen your internal documents, recent news, or private data.

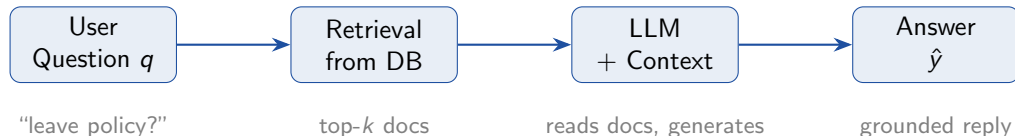
[OK] The RAG Idea

Before answering, **look it up first**.

Retrieve relevant documents, then give them to the model as reference material.

RAG = Retrieval-Augmented Generation

RAG gives a language model a **reference library it can consult** before answering any question.



Step 1 — Text Must Become Numbers

Computers cannot process language directly. We need a map:

$$f : \text{Text} \longrightarrow \mathbb{R}^d$$

This map f is called an **Embedding model** (typically $d = 768$ or 1536).

Examples (schematic, $d = 4$ for illustration)

$$f(\text{"security deposit"}) = (0.12, -0.83, 0.44, 0.71)$$

$$f(\text{"rental guarantee"}) = (0.11, -0.81, 0.46, 0.69)$$

$$f(\text{"photosynthesis"}) = (-0.67, 0.23, -0.91, 0.05)$$

Key property: semantically similar texts map to **nearby vectors**.

Step 2 — Measuring Semantic Similarity

Given two text vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, we measure similarity via **cosine similarity**:

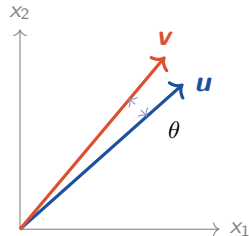
$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \in [-1, 1]$$

Why cosine, not Euclidean distance?

We care about **direction** (semantic content), not **magnitude** (which is affected by text length).

Cosine similarity is invariant to scaling:

$$\text{sim}(\mathbf{u}, c\mathbf{v}) = \text{sim}(\mathbf{u}, \mathbf{v}), \quad c > 0$$



Retrieval = Nearest Neighbour Search

Setup. A corpus of N documents, each embedded as a vector:

$$\mathcal{D} = \{\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_N\} \subset \mathbb{R}^d$$

A user query q is also embedded: $\mathbf{q} = f(q) \in \mathbb{R}^d$.

Retrieval problem (top- k):

$$\text{Find } \mathbf{d}_{(1)}^*, \dots, \mathbf{d}_{(k)}^* = \arg \max_{\mathbf{d}_i \in \mathcal{D}} \text{top-}k \text{ sim}(\mathbf{q}, \mathbf{d}_i)$$

This is exactly **k -Nearest Neighbour (k -NN) Search** in $\mathbb{R}^d$ under cosine similarity.

Why Brute Force Fails — and ANN to the Rescue

Brute-force cost

Compute $\text{sim}(\mathbf{q}, \mathbf{d}_i)$ for all i :

$O(N \cdot d)$ FLOPs per query

For $N = 10^7$, $d = 1536$:

$\approx 1.5 \times 10^{10}$ FLOPs

Too slow for production.

Approximate NN (ANN)

HNSW (Hierarchical Navigable Small World):

- Builds a multi-layer graph over all vectors
- Upper layers: sparse “highways”
- Lower layers: dense “local roads”
- Query: descend from top, refine locally

Complexity: $O(\log N)$ — used by FAISS, Pinecone, Chroma

Trade-off: ANN is not guaranteed to find the global optimum, but the approximation is sufficient in practice.

How Is the Embedding Map f Learned?

f is a **Transformer neural network** trained with **Contrastive Learning**.

Training signal. For each query q :

- d^+ : a **relevant** (positive) document
- $d_1^-, \dots, d_m^-$: **irrelevant** (negative) documents

Objective. Pull positive pairs together, push negatives apart:

$$\text{maximise } \text{sim}(f(q), f(d^+)) \quad \text{while} \quad \text{minimise } \text{sim}(f(q), f(d_j^-))$$

The InfoNCE Loss

The standard loss function is **InfoNCE** (a contrastive cross-entropy):

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{q}, \mathbf{d}^+)/\tau)}{\sum_{j=0}^m \exp(\text{sim}(\mathbf{q}, \mathbf{d}_j)/\tau)}$$

Components

Symbol	Meaning
$\mathbf{q}$	embedded query
$\mathbf{d}^+$	embedded positive document
$\mathbf{d}_j$	positive + all negatives in batch
$\tau > 0$	temperature — controls distribution sharpness

From Retrieved Docs to an Answer

After retrieval we have top- k documents $d_{(1)}^*, \dots, d_{(k)}^*$.

We construct a **prompt**:

$$P = [d_{(1)}^*, d_{(2)}^*, \dots, d_{(k)}^*, q]$$

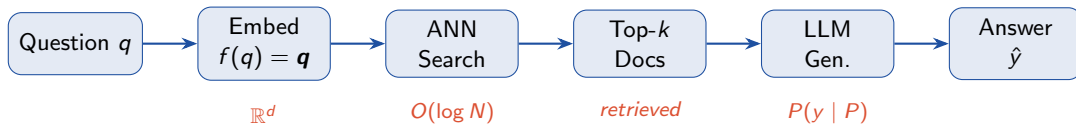
The language model then produces the answer by **conditional language modelling**:

$$\hat{y} = \arg \max_y P(y \mid d_{(1)}^*, \dots, d_{(k)}^*, q)$$

In practice, the model autoregressively samples token by token:

$$P(y_1, \dots, y_T \mid P) = \prod_{t=1}^T P(y_t \mid y_{<t}, P)$$

The Full RAG Pipeline



Step	Operation	Math
Embedding	Text $\rightarrow$ vector space	$f : \text{Text} \rightarrow \mathbb{R}^d$
Similarity	Distance in $\mathbb{R}^d$	Cosine similarity
Retrieval	Approximate k -NN	HNSW, $O(\log N)$
Training f	Contrastive learning	InfoNCE loss
Generation	Conditional LM	Autoregressive $P(y P)$

Where Are the Open Problems?

The basic pipeline is simple.

Making it fast and accurate is where research lives.

Challenge	Description
Retrieval quality	Query and answer are phrased differently
Context utilisation	LLM ignores relevant content placed mid-prompt
Multi-step queries	One retrieval step is insufficient
Chunking strategy	How to split documents into retrievable units
Knowledge structure	Flat vector store loses relational knowledge
Evaluation	How to measure RAG quality automatically

Research Direction 1: Retrieval Quality

Problem. The user's query and the stored answer are phrased differently. Their embeddings may be far apart despite semantic equivalence.

Query Rewriting

Use an LLM to rephrase q into a form closer to how answers are written.

$$q \xrightarrow{\text{LLM}} q' \xrightarrow{f} \mathbf{q}'$$

HyDE (Hypothetical Doc Embedding)

Generate a *hypothetical* answer $\hat{d}$ and use $f(\hat{d})$ as the query vector.

$$q \xrightarrow{\text{LLM}} \hat{d} \xrightarrow{f} \mathbf{\hat{d}}$$

Retrieval now happens in “answer space”.

Better Embedding Models

Improve the contrastive training of f itself — larger batches, harder negatives, domain-specific fine-tuning.

Research Direction 2: Context Utilisation

Problem. Even if relevant documents are retrieved, the LLM may not use them correctly.

“Lost in the Middle” Phenomenon (Liu et al., 2023)

Performance degrades when relevant content is placed in the **middle** of the prompt. Models are biased towards the beginning and the end.

Re-ranking

After retrieval, use a cross-encoder model to **re-score and reorder** results, placing the most relevant content first.

Context Compression

Summarise or filter retrieved documents: pass only the **essential sentences** to the LLM, reducing noise.

Research Direction 3: Multi-Step Retrieval

Problem. Complex queries require information from *multiple sources* that must be combined.

Example

“Compare the 2024 financial performance of Company A and Company B.”
⇒ requires querying A’s report, then B’s report, then synthesising.

Multi-hop Retrieval

Decompose into sub-questions.
Each hop retrieves one piece.

Iterative Retrieval

Interleave retrieval with generation:
retrieve → generate →
retrieve again.

Agentic RAG

LLM decides *when* and *what*
to retrieve dynamically.
Retrieval as a **tool call**.

Research Direction 4: Chunking Strategy

Problem. Documents must be split into retrievable units (chunks). Chunk size is a critical and poorly understood hyperparameter.

Strategy	Idea	Trade-off
Fixed-size chunking	Split every n tokens	Simple but context-breaking
Semantic chunking	Split at semantic boundaries	Better coherence, harder to implement
Hierarchical chunking	Small chunks for retrieval, large chunks for generation	Best of both worlds, more complex pipeline
Late chunking	Embed full doc, then split	Preserves global context

Research Direction 5: GraphRAG

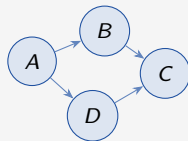
Problem. A flat vector store treats all documents as independent. Real-world knowledge has **structure** (hierarchies, dependencies, causality).

Standard RAG



independent vectors

GraphRAG (Microsoft, 2024)



knowledge graph

GraphRAG first extracts entities and relations from documents, then traverses the graph during retrieval.

Particularly powerful for **global** or **summarisation-type** queries.

Research Direction 6: Evaluation

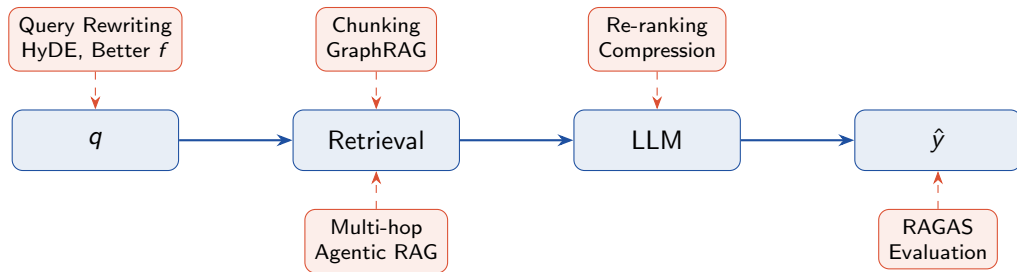
Problem. How do we know if a RAG system is actually good?

RAGAS Framework — Key Metrics

Metric	Definition
Faithfulness	Is the answer grounded in the retrieved documents? (No hallucination)
Answer Relevancy	Does the answer address the actual question?
Context Recall	Were all necessary documents retrieved?
Context Precision	Are retrieved documents actually useful?

Open challenge: using LLMs to evaluate LLM outputs introduces systematic bias. Constructing reliable, unbiased benchmarks is an active research area.

Research Landscape — Summary



Every arrow hides unsolved problems — each is a potential contribution.

Key Takeaways

- ① **RAG** = retrieval + generation. Give the model a reference library at inference time.
- ② **Embeddings** map text into $\mathbb{R}^d$. Semantic similarity becomes *cosine similarity*.
- ③ **Retrieval** is k -NN search in $\mathbb{R}^d$, approximated by HNSW for scalability.
- ④ **Embedding models** are trained via *contrastive learning* (InfoNCE loss).
- ⑤ **Generation** is conditional language modelling: $P(y \mid \text{docs}, q)$.
- ⑥ **Open problems** span every stage of the pipeline — retrieval quality, chunking, multi-hop reasoning, knowledge structure, and evaluation.